

從「生成程式」到「修復程式」：我如何讓邊緣 AI 的數學出題更可靠

Scaffolding Prompt × AST Active Healer × Ablation Study

「在整體研究的三條技術路線中，我持續負責的是程式生成可靠性這一線。這條線教會我的東西，其實比技術本身更重要。」

MY ROLE | RESEARCH OWNERSHIP
程式生成可靠性技術線Scaffolding Prompt · Domain Function · AST Active Healer · Ablation Study · Re-designed Validation · Error Analysis
在團隊的多條研究技術線中，我持續負責程式生成可靠性相關方法、實作與驗證。

RESEARCH CONDITIONS 本研究未接受大學實驗室或校外研究單位指導；程式開發、實驗設計與數據分析由本人完成，過程中使用生成式 AI 工具輔助資料查找與概念釐清，並與家人討論研究方向。

這份文件記錄的，是一個從「想省錢」開始、最後學會「誠實面對結果」的過程：



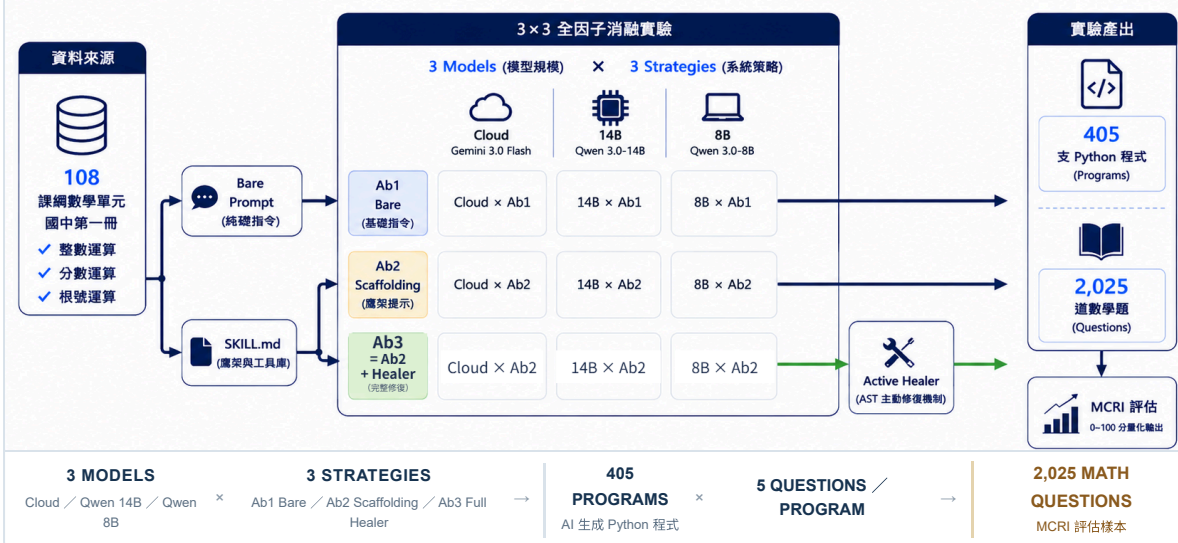
這是我第一次真正學著做研究——包括學著接受，一個誠實的小數字，比一個漂亮的大數字更有價值。

父親曾提醒我，出題系統每次生成程式都需要呼叫雲端 AI，長期 API 用量會累積成本。回頭檢視系統累積的生成紀錄後，我發現：同一個數學單元中，程式結構高度相似，錯誤位置也具有重複性。這讓我開始思考：既然錯誤有規律，是否將重複性的工作抽離，讓模型不用每次都重新生成整支程式？

為系統性驗證這個想法，我設計一組 3×3 全因子消融實驗：三種模型規模交叉三種系統策略，共產出 405 支 AI 生成的 Python 程式，並由每支程式生成 5 道題，共 2,025 道數學題進行 MCRI 評估。

3×3 全因子實驗設計

模型規模 × 系統策略 = 全面探討 Scaffolding 與 Healer 的交互作用



「鷹架」怎麼運作？我把重複性的工作從模型任務中抽離——從辨識數學特徵、抽離可變參數，到組成結構化規格書，讓模型只需專注邏輯轉譯。

鷹架提示生成與注入流程圖（完整流程）



能不重新生成，就不要重新生成。

即使加入鷹架，AI 生成的 Python 程式仍可能出現格式、語法、結構與重複宣告等錯誤。因此我設計四層依序運作的 Active Healer，在重新生成整份程式之前，先處理可辨識的局部問題。

AST Active Healer 主動修復機制運作流程

從髒亂的 AI 生成程式碼，透過四層修復機制，自動輸出可執行且乾淨的程式碼



修好程式，只代表它可以執行；但「能執行」不代表生成的數學內容正確。因此初版研究把評估拆成兩個面向：程式本身的可靠性，以及生成數學內容的品質，建立 MCRI 作為量化依據。

Compile / Execute
≠ Mathematical Correctness

MCRI 雙軌評估體系 Math Code Reliability Index

總分 100 POINTS

程式價值 PROGRAM VALUE	50%	數學價值 MATH VALUE	50%
30% 工程基石 + 資料衛生 Syntax / Execution / Format 驗證語法正確性、執行效能與格式完整性		20% 數學正確性 Mathematical Correctness 使用 SymPy 進行外在強健性驗證，確認答案正確	
20% 程式架構品質 Architecture Quality 使用 AST 進行靜態分析，評估程式架構品質		30% 教學有效性 Teaching Effectiveness 評估解題推導深度與數學運算廣度	

程式價值 (50%) + 數學價值 (50%) = MCRI 總分 (0-100 分)

無需標準答案的綜合評估，適用於自動生成程式的品質評估。

KEY TERMS 本研究關鍵術語 後續頁面沿用以下縮寫與術語

AST

Abstract Syntax Tree

將程式轉為語法樹，以結構方式定位與修改程式。

Active Healer

AST Active Healer

本研究設計的規則式主動修復機制。

Scaffolding

Scaffolding Prompt

將題型規則、參數與流程結構化後提供給模型。

Domain Function

Domain-specific Function

將重複數學運算封裝為可重複呼叫的工具。

MCRI

Math Code Reliability Index

初版研究自訂的程式可靠性與數學品質綜合指標。

Ablation

Ablation Study

加入或移除系統元件，比較各元件帶來的增量效果。

METHOD COMPLETE

Active Healer + MCRI

Healer 負責「修」，MCRI 負責「量」。到這裡，生成、修復與評估的流程都已經建立完成。

NEXT

我怎麼驗證它？

INITIAL EXPERIMENT 初版實驗是怎麼跑的？

01

課本例題

108 課綱教材 | 整數／分數／根號

從課本例題與解析取得
題型結構與解題步驟

02

建立鷹架

Scaffolding Prompt | Domain
Function整理可變參數、解題流
程與允許使用的函數

03

AI 重新生成

簡單／中等／困難

要求模型依據這套鷹架
產生不同難度的新題目
與程式

04

Active Healer

四層依序修復

修復生成程式中的格
式、語法與結構問題

05

MCRI 評分

Program Value + Math Value

使用研究中自行設計的
MCRI 評估生成結果

INITIAL HYPOTHESIS 初版驗證假設

01 跨模型

Cloud / Qwen 14B / Qwen 8B

如果不同模型都改善，代表不是單一模型的偶然。

02 跨題型

整數 / 分數 / 根號

如果不同數學領域都改善，代表不是只對特定題型
有效。

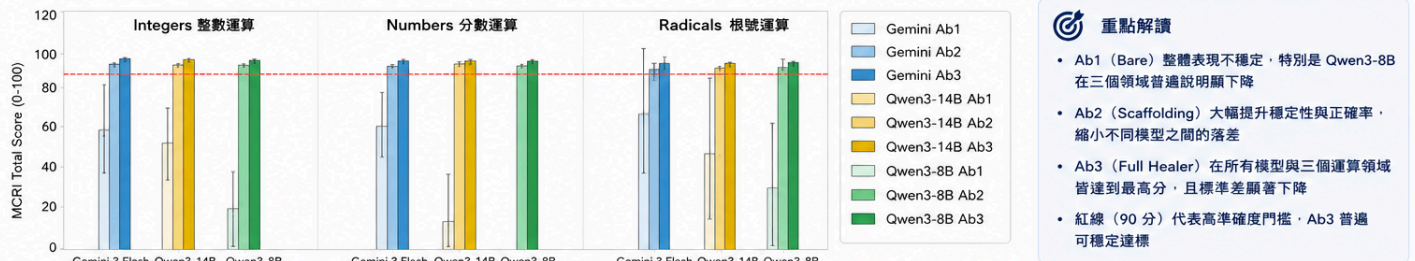
03 跨策略

Bare / Scaffolding / Full Healer

如果 Ab3 更高、更穩定、更省資源，當時我認為方法
有效。

當時我的想法很直接：如果三種模型、三個運算領域都得到一致結果，就足以證明這套方法有效。

附圖 1：整數、分數、根號四則運算消融實驗總體效能得分結果比較



附表 1：各系統策略之 MCRI 評分描述性統計與差異檢定

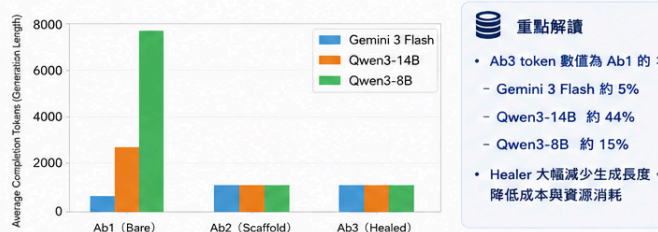
組別	平均數	標準差	Kruskal-Wallis 總體檢定	事後檢定與效果量
Ab1 (Bare)	41.00	34.03	H = 343.40, $p < .001$	
Ab2 (Scaffolding)	94.39	1.96		Ab2 > Ab1 ($p < .001$)
Ab3 (Full Healer)	97.41	1.67		Ab3 > Ab2 ($p < .001$, $d = 1.66$)

重點解讀

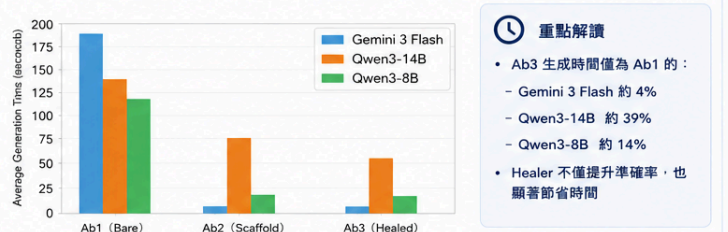
- Ab3 相較 Ab1 提升 56.41 分，標準差從 34.03 收斂到 1.67
- 統計檢定結果顯示三組差異極顯著 ($p < .001$)
- 效果量 $d = 1.66$ 屬「極大效果」，代表 Healer 帶來實質且穩定的提升

附圖 2 與 3：雲端模型與邊緣 AI 在分數四則運算的效率比較

附圖 2：雲端模型與邊緣 AI 在分數四則運算的 token 數比較



附圖 3：雲端模型與邊緣 AI 模型在分數四則運算的生成時間比較



整體結論

在 MCRI 這套評估標準下，Ab3 同時取得最高分數、最低標準差，並大幅降低 token 與生成時間。
當時我認為，這已經足以說明方法有效。數據很漂亮，統計也顯著。
但這些數字，全都是用我自己設計的尺量出來的。

AT THAT TIME

當時我的判斷

41.00 → 94.39 → 97.41 | SD 34.03 → 1.67 | 跨模型、跨運算領域呈現相近趨勢

我第一次覺得：我可能真的把 AI 出題不穩定的問題解決了。

數據漂亮、統計顯著，而且結果看起來更穩定。當時的我，幾乎沒有理由懷疑這套方法。

97.41

SD 1.67 | $p < .001$ | $d = 1.66$ I THOUGHT
IT WORKED.

NEXT | 評審看完後，問了什麼？

LOOK AGAIN | 重新看一次初版實驗

如果把 97.41 暫時放到一旁，這套實驗究竟是怎麼得到這個分數的？

當時我看到的是一條完整的生成與評估流程；重新拆開每個步驟後，我才開始注意到原本沒有追問的問題。



THE QUESTIONS BEHIND 97.41 重新看過流程後，我開始追問三件事

01 測試資料真的獨立嗎？

我從課本例題整理題型規則，再要求 AI 依照這些規則生成簡單、中等、困難題目。重新檢視後我開始懷疑：這些測試題雖然是新生成的，是否真的足以作為獨立於生成方法之外的測試？

TEST INDEPENDENCE

02 這把尺，真的足夠嗎？

97.41 來自研究中自行建立的 MCRI。它同時衡量程式可靠性與數學品質，但高 MCRI 是否就足以代表真正的生成正確性與方法有效？漂亮的分數，仍需要更獨立的驗證依據。

VALIDATION / BENCHMARK

03 分數提升，真的代表方法有效嗎？

消融結果其實已經把兩階段效果拆開：

Ab1 → Ab2 +53.39 Scaffolding 主要提升	Ab2 → Ab3 +3.02 Healer 增量改善
--	--

真正的問題不是「誰造成提升」，而是：這些提升在獨立資料與更嚴格驗證下，是否仍然成立？

GENERALIZATION / EXTERNAL VALIDATION

初版創意說明書

tinyurl.com/3w3wprxf



評審建議原文

tinyurl.com/47y2prhe



而旺宏評審看完這些結果後，追問的，正好也是這些問題。

原本我以為評審會關心 97.41 有多高；但他們真正關心的是——這個 97.41，是在什麼條件下得到，又能證明什麼？

REVIEWER FEEDBACK 旺宏評審進一步指出

01 聚焦真正的原創貢獻

作品橫跨多個子系統，雖然都能運作、也都有數據，但驗證深度不足；其中**最具實質貢獻者為 AST Active Healer**。

FOCUS / ORIGINAL CONTRIBUTION

→ p.6-7 | 固定規則修復與能力邊界分析

02 建立更有說服力的驗證

評估方式必須清楚揭露，並優先考慮標準 Benchmark、公開資料、真實回饋或外部驗證，而不是只依賴自行建立的評估方式。漂亮的分數，不等於方法已被充分證明。

VALIDATION

→ p.5 | Rule Development / Held-out Evaluation 分離

03 建立公平的比較條件

不同模型接受的 Prompt、修復流程與處理條件若不一致，就不能直接把結果解讀成模型能力的反超。比較必須建立在更一致、更公平的控制條件上。

FAIR COMPARISON / CONTROL

→ p.5 | Prompt Frozen / Healer Rules Frozen

04 把 Healer 的增量效果驗證得更深

初版 Ab2 → Ab3 已經提供 Healer 的消融比較，但評審期待的不只是「分數再提高」，還需要回答：Healer 實際修復了哪些錯誤？哪些錯誤不能修？在固定規則與獨立測試下，效果是否仍然成立？

ABLATION / ERROR ANALYSIS

→ p.6-7 | Original FAIL → Modified → PASS / FAIL

RESPONSE TO REVIEWERS 評審意見如何轉化成實際的實驗修改

01 | FOCUS

聚焦原創貢獻

→ 整份決賽驗證只聚焦 AST Active Healer

02 | INDEPENDENT EVALUATION

建立更獨立的驗證

→ Rule Development / Held-out 資料切分

03 | FAIR COMPARISON

建立公平比較條件

→ Frozen Prompt / Frozen Healer Rules

04 | ERROR BOUNDARY

深入分析修復能力

→ 逐一追蹤 Original FAIL → Modified → PASS / FAIL

REVIEWER
DIRECTION聚焦 **AST Active Healer**，把真正的原創貢獻做深。

Held-out Evaluation × 公平對照 × 固定規則 × 修復能力邊界

97.41 沒有消失， $p < .001$ 也沒有變錯。

真正改變的，是我對「證明」這件事的理解。

初版已經有消融、有統計，也看得到 Scaffolding 與 Healer 各自帶來的增量改善；但我開始明白：「分數提升」與「方法被充分證明」，其實是兩件不同的事。

97.41

↓
?

NEXT | 重新設計實驗

97.41 之後，我重新驗證：這套系統遇到「沒看過的題目」，還能工作嗎？

這一次我不再追求漂亮的分數，而是把題目、評分方式與修復規則重新限制，直接追蹤每一個原始 FAIL 經 Healer 後究竟發生什麼。

決賽成果報告書

tinyurl.com/mr3n2rnr



01 把原本的「尺」換掉：先切資料，再凍結規則

DATA SOURCE 題目來自公開之國中教育會考試題，非由本研究生成，作為獨立於開發流程之外部資料來源。

決賽重新設計 | 把評審的疑問，直接改寫成實驗條件

不再只看「分數有沒有變高」，而是重新控制題目、Prompt、資料切分與評估方式。

01 四種 Prompt 條件：把不同支援拆開比較

Ab1

僅原始題目

基礎對照；不提供額外程式規格或領域工具。

Ab2g

原始題目＋一般程式規格

提供不依數學領域區分的一般程式與輸出規格。

Ab2d+api

原始題目＋領域 API 鷹架

提供预先建立的 Domain Functions/API 規格與呼叫方式。

Ab2d+spec

原始題目＋題目專屬規格鷹架

依題目提供更明確的程式骨架、生成規則與流程限制。

02 Math16 正式重測：先切資料，再凍結規則

16 題

Math16

3 模型

Qwen3.5 9B / 4B
+ Gemini 3.5 Flash

4 Prompt

Ab1 / Ab2g
Ab2d+api / Ab2d+spec

5 Seeds

固定隨機種子

960 格

總實驗單元

4 題規則開發題 | 240 格

用來設計 Prompt、分析錯誤、建立 AST Healer 規則

規則狀態：可依開發結果調整

RULES
FROZEN

12 題保留評估題 | 720 格

只用來檢驗固定 Prompt 與固定 Healer 規則是否有效

規則狀態：正式評估前全部凍結

這個改動回答評審最重要的問題：

正式評估題不再參與 Prompt 或修復規則的開發；避免看到答案後再回頭把規則修到會過。

02 Math16：真正把規則開發與測試分開

16

會考數學題
涵蓋 4 個數學領域

3

Models

4

Prompt Conditions

5

Fixed Seeds

960

Experimental Units

4 Questions | RULE DEVELOPMENT

240 units

12 Questions | HELD-OUT EVALUATION

720 units

240 + 720 = 960

~ PROMPT FROZEN

~ HEALER RULES FROZEN

PASS = 程式正確執行 + 數學答案正確；正式 Held-out Evaluation 開始後，不得再依測試結果修改 Prompt 或 Healer 規則。

03 初版驗證 → 決賽重新驗證：到底改了哪些條件？

FROM INITIAL STUDY TO RE-DESIGNED VALIDATION

項目	INITIAL STUDY 初版	RE-DESIGNED VALIDATION 決賽
題目 / 測試來源	依原研究題型與生成流程進行驗證	Math16 16 題會考數學題 (涵蓋 4 個數學領域)
資料切分	未做 Rule Development / Held-out 的明確切分	4 題 Rule Development + 12 題 Held-out Evaluation
模型	Cloud / Qwen 14B / Qwen 8B	Gemini 3.5 Flash / Qwen 3.5 9B / Qwen 3.5 4B
評估方式	MCRI 0-100 分	PASS / FAIL (執行正確且答案正確才算 PASS)
修復規則	原始 Active Healer 設計	Frozen Rules + Answer-blind：能確定才修，不確定即棄權
比較單位	組別平均表現與消融差異	逐一追蹤每個 Original FAIL 的最終去向
主要結果	97.41 SD 1.67	9 / 342 = 2.63% Safe Recovery Rate

MODEL UPDATE 重新驗證階段改採新的模型組合，並重新建立一致的 Prompt、Seed 與修復條件；初版與重新驗證結果不直接混合作為同一批實驗比較。

更重要的是：我把 Healer 的權力縮小了。

如果 Healer 可以重新理解題目、重算答案，那麼「修復成功」根本無法證明是修復規則有效。因此決賽版採用 answer-blind 設計：只走固定的 Tier A → B → C → D 規則 → p.6 | 四層修復規則，能確定才修；不能確定，就棄權。

~ PROMPT FROZEN

~ HEALER RULES FROZEN

NEXT | Healer 到底能修多少？

Healer 在決賽版被重新限制：能確定才修，不能確定就棄權。

Answer-blind：Healer 不知道正確答案。只有原始 FAIL 才進入修復，PASS → PASS 不計為救援；修復後使用完全相同的測試重新評估，只有 FAIL → PASS 才算「成功救援」。

AST Healer 在決賽版被重新限制：能確定才修，不能確定就棄權

評估不再問「Healer 有沒有讓總分變高」，而是直接追蹤每個原始 FAIL 經修復後究竟變成什麼。

01 答案盲 (answer-blind) 的修復與重新評估流程



02 Healer 只能修「能確定」的內容：固定規則與介入原則

固定四層 Tier 規則 (逐層檢查)			Healer 的介入原則：修不到就棄權，不猜答案	
			Healer 可以修 (在固定規則下可確定規則下可唯一判定)	Healer 不修 (超出確定性修復範圍)
Tier A 保守修復	僅處理局部結構可唯一判定的錯誤 (介面、資料流、封裝、格式錯誤)	安全修復	✓ 介面 / 資料流錯誤 (變數未定義、型別不符)	✗ 數學答案錯誤
Tier B 結構修復	擴充表面語法與結構修復 (符號缺失、括號、空程式區塊、綁定)	安全修復	✓ 結構錯誤 (括號、符號缺失、空程式區塊)	✗ 核心演算法錯誤
Tier C API 修復	領域 API 函式名稱或參數錯誤修復 (只能依固定規格判定的錯誤)	安全修復	✓ 領域 API 使用錯誤 (函式名稱、參數位置)	✗ 題意理解錯誤
Tier D 重綁定	移除同名遮蔽或重複定義後重新綁定 (無法唯一選擇時直接棄權)	有條件修復	✓ 同名遮蔽或重複定義 (可唯一判定的情況)	✗ 需要重新設計流程或邏輯
			✓ 固定格式錯誤 (輸出結構、封裝方式)	✗ 沒有唯一確定修法的模糊情況



Healer 的價值：只修自己有把握修對的錯誤，不做可能讓答案變對的猜測。
這樣才能確保修復結果的可信度，並建立公平、可重現且可檢驗的實驗基準。

SAFE TO REPAIR | 可修

Interface | 介面
Data Flow | 資料流
Local Structure | 局部結構
Fixed-format | 固定格式錯誤

只能修「由固定規格唯一判定」的錯誤。

OUT OF SCOPE | 不應由 Healer 修

Mathematical Answer
數學答案
Core Algorithm
核心演算法
Problem Understanding
題意理解

需要重新理解題目，就不應由固定規則 Healer 猜測。

Healer 的價值不是「把所有錯誤修好」，而是「只修自己有把握修對的錯誤」。

720

HELD-OUT UNITS

342

ORIGINAL FAIL

42

HEALER MODIFIED

9

FAIL → PASS

2.63%

安全救援率
FAIL → PASS / ORIGINAL FAIL
= 9 / 342

重新驗證後，Healer 真正安全救回的只有 9 格。

這個結果遠沒有初版的 97.41 漂亮，但它第一次讓我真正看見 Healer 的能力邊界。

陌生題型

→ Domain Function Coverage

不足

→ 模型找不到現成工具

→ Edge Model 必須自己建立運

算邏輯

→ 答案 / 演算法錯誤增加

→ Answer-blind Healer 無法安全介

問題可能不是「Healer 修不好」，而是前面的模型根本沒有拿到足夠的工具。

NEXT QUESTION

如果真正的瓶頸是 Domain Function Coverage，那麼把 Math16 所需工具補齊之後，生成成功率應該會提高。

這就是下一頁要驗證的假設。

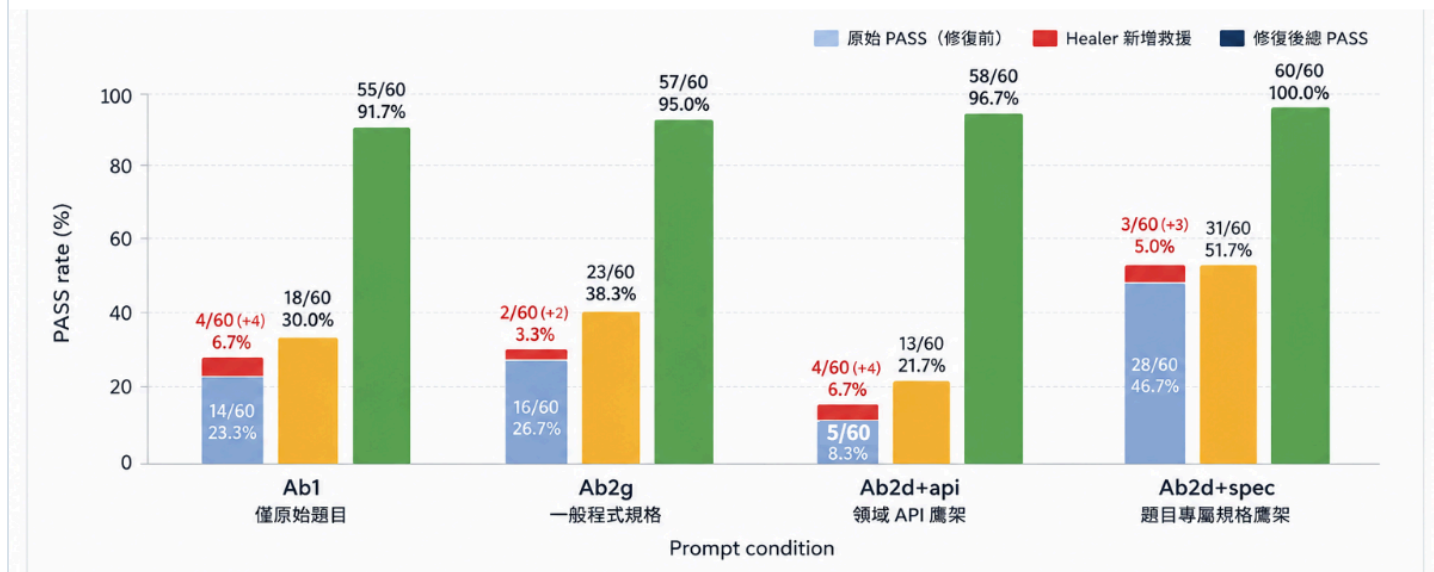
2.63%

WHY?

先看 Healer 究竟救了哪些失敗，再看這些失敗分佈在哪些層級。

Healer 新增救援 (FAIL → PASS)

同一批 720 個實驗單元，在 Healer 修復後新增的救援量（紅色），堆疊於原始 PASS（淺色）之上。



OBSERVATION 01

Gemini 3.5 Flash 在四種提示條件下都維持極高原始通過率。

OBSERVATION 02

Qwen 3.5 4B / 9B 對 Prompt 與工具條件非常敏感。

OBSERVATION 03

Ab2d+api 並沒有自然優於其他條件，顯示「有工具」不等於「知道怎麼用工具」。

工具存在 ≠ 工具能被正確使用。

FAILURE LEVEL 重新驗證階段使用的錯誤分類，與前頁 MCRI 評估構面不同。

L0 | 系統或基礎設施失敗 L1 | 程式擷取、解析或語法失敗 L2 | 格式、函式入口或輸出包裝 L3 | Domain API / 工具使用失敗 L4 | 執行、組裝或資料流失敗 L5 | 答案或核心演算法錯誤

各失敗層級的原始 FAIL 與 Healer 救援結果

Healer 成功救回 9 格，原始 FAIL 救援率為 2.63% (9/342)。

失敗層級	主要錯誤類型	原始 FAIL (格)	成功救援 (格)	救援率 (成功救援 / 原始 FAIL)
L0	系統或基礎設施失敗	0	0	—
L1	程式擷取、解析或語法失敗	114	1	0.88%
L2	格式、函式入口或輸出包裝失敗	22	6	27.27%
L3	領域 API 或工具使用失敗	3	0	0%
L4	執行、組裝或資料流失敗	59	2	3.39%
L5	答案或演算法錯誤 (核心邏輯錯誤)	144	0	0%
合計		342	9	2.63%

註：救援率 = 該層成功救援數 ÷ 該層原始 FAIL 數；L0 沒有原始 FAIL，因此不計算救援率。

BOUNDARY, NOT FAILURE

342 個原始 FAIL 中，Healer 成功救回 9 格 (2.63%)：主要集中於 L2 | 格式、函式入口、輸出包裝 (6 格) 與 L4 | 執行、組裝、資料流 (2 格)，另有 L1 (1 格)；L5 | 答案或演算法錯誤共 144 格 → 0 格救援。

L1 (1) + L2 (6) + L4 (2) = 9 格

這不是 Healer 「太弱」，而是證明 answer-blind Healer 沒有越界去猜答案。

如果真正的瓶頸是「工具覆蓋率」，把 Domain Function 補齊後，結果會改變嗎？

ORIGINAL DOMAIN API
依原本課本題型建立

MATH16 COVERAGE GAP
部分題型缺少所需工具

COMPLETE DOMAIN API
補齊 Math16 所需 Domain Functions

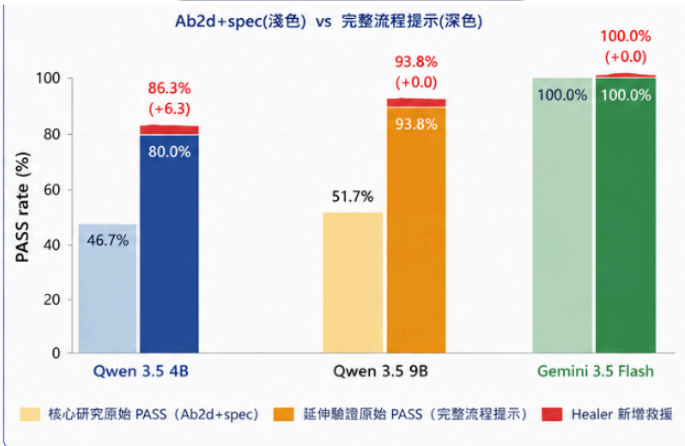
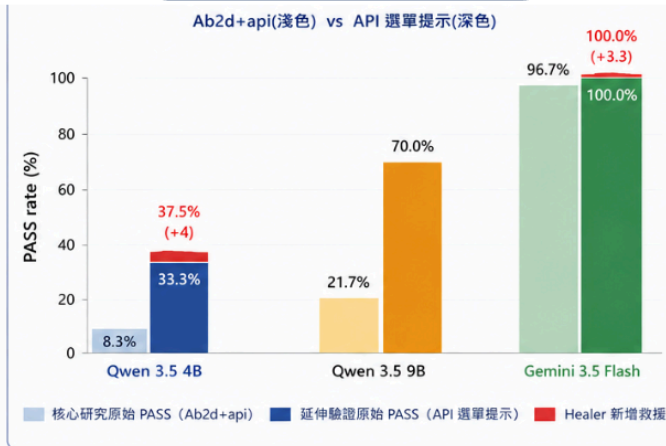
這不是修改 Held-out Evaluation 的結果，核心研究的 2.63% 保持不變。這是一組新的目標性延伸驗證，用來回答另一個問題：工具覆蓋率與流程提示，是否會影響模型生成成功率？

原始 PASS 與 Healer 新增救援（延伸驗證結果）

補齊 16 題所需的領域 API，分別比較「API 選單提示」與「完整流程提示」在原始 PASS 與 Healer 新增救援上的表現。

A. API 選單提示（僅提供工具清單）

B. 完整流程提示（提供使用方法）



原始 PASS 提升效果（延伸驗證 vs 核心研究）

模型	API 選單提示	完整流程提示
Qwen 3.5 4B	+24.17 pp	+33.33 pp
Qwen 3.5 9B	+48.33 pp	+42.08 pp
Gemini 3.5 Flash	+3.33 pp	+0.00 pp

pp = percentage points (百分點)



Healer 新增救援（僅針對原始 FAIL）

模型	API 選單提示	完整流程提示
Qwen 3.5 4B	+4 格 (共 16 格)	+5 格 (共 21 格)
Qwen 3.5 9B	+2 格 (共 5 格)	+0 格 (共 24 格)
Gemini 3.5 Flash	+3.3%	+0%

註：Healer 僅處理可由固定規格判定的局部錯誤。



整體觀察重點

- 補齊領域 API 後，原始 PASS 明顯提升，證實工具覆蓋不足是主要限制。
- 完整流程提示的提升效果普遍優於僅提供工具清單。
- Gemini 3.5 Flash 在兩種提示下均接近滿分，目標題型對其較為友善。
- Healer 的新增救援集中於可由固定規格修復的局部錯誤，不處理答案或核心演算法問題。

註：API 選單提示提供「有哪些工具」；完整流程提示進一步明確「如何使用、參數來源、呼叫順序、中間資料流與答案組裝方式」。

KEY FINDING 01 | TOOL COVERAGE

補齊領域 API 後，兩個 Qwen 模型原始 PASS 大幅提升，Gemini 影響有限。

KEY FINDING 02 | PROCESS SCAFFOLDING

提供完整流程說明，對兩個 Qwen 模型皆帶來明顯提升，顯示「如何使用工具」與「有哪些工具」同樣重要。

KEY FINDING 03 | WHAT ACTUALLY CHANGED?

不是讓 Healer 變得更會猜，而是模型知道：① 有哪些工具 ② 參數從哪裡取得 ③ 如何呼叫 ④ 呼叫順序 ⑤ 資料如何流動 ⑥ 答案如何組裝。

TOOL COVERAGE × PROCESS SCAFFOLDING → GENERATION RELIABILITY

SYSTEM IMPLICATION 由實驗結果推導的系統設計方向，非本次 held-out 實驗直接驗證的結論。

NEW / UNKNOWN | 第一次遇到的新題型

STRONGER LLM

理解新問題 → 建立 Domain Function → 建立 / 更新 Scaffolding → 累積成可重複使用的系統能力

HIGHER ONE-TIME COST

KNOWN / REPEATABLE | 已經處理過的題型

EDGE MODEL + DOMAIN FUNCTIONS + SCAFFOLDING + SAFE HEALER

重複生成 → 問題拆分 → 不同教材重新組合 → 本地執行

LOWER REPEATED INFERENCE COST

Cloud + Edge：不是讓 Edge AI 取代大型模型，而是不要讓大型模型一直重做已經解決過的事。

K-12 數學受到課綱、單元與知識範圍約束，相較開放領域，其核心運算需求具有較高的可累積性：NEW PROBLEM → Stronger LLM 理解 → 建立 / 更新 Domain Function → 加入 Domain Library → Edge Model 可重複使用 → 下一次降低生成成本。

我真正想建立的，不是一個永遠依賴昂貴模型的生成器，而是一套把「第一次解決問題的成本」轉化成「下一次可以重複使用的能力」的系統。

TERMS & ABBREVIATIONS 術語速查

AST Abstract Syntax Tree 抽象語法樹，以程式結構而非純文字分析程式碼

Held-out Held-out Evaluation 規則凍結後，不參與開發的獨立測試資料

pp percentage point 百分點

MCRI Math Code Reliability Index 初版研究使用的程式與數學品質綜合指標

Answer-blind Healer 不知道正確答案，只依固定規格修復

SD Standard Deviation 標準差

Ab Ablation Condition 消融實驗條件 (Ab1 / Ab2 / Ab3)

API Application Programming Interface 模型可呼叫的功能介面

p / d p-value / effect size 統計顯著性指標 / 效果量

WHAT I ACTUALLY LEARNED

97.41，讓我以為自己已經把問題解決了。

2.63%，讓我第一次真正知道它能解決多少。

真正的研究，不是把數字做到最好看，而是知道這個數字究竟證明了什麼。

研究一開始，我只是想讓模型少浪費一些 Token；最後我學到的，卻是如何把第一次解決問題的經驗變成可以重複使用的工具，同時誠實地知道系統做到什麼、又做不到什麼。